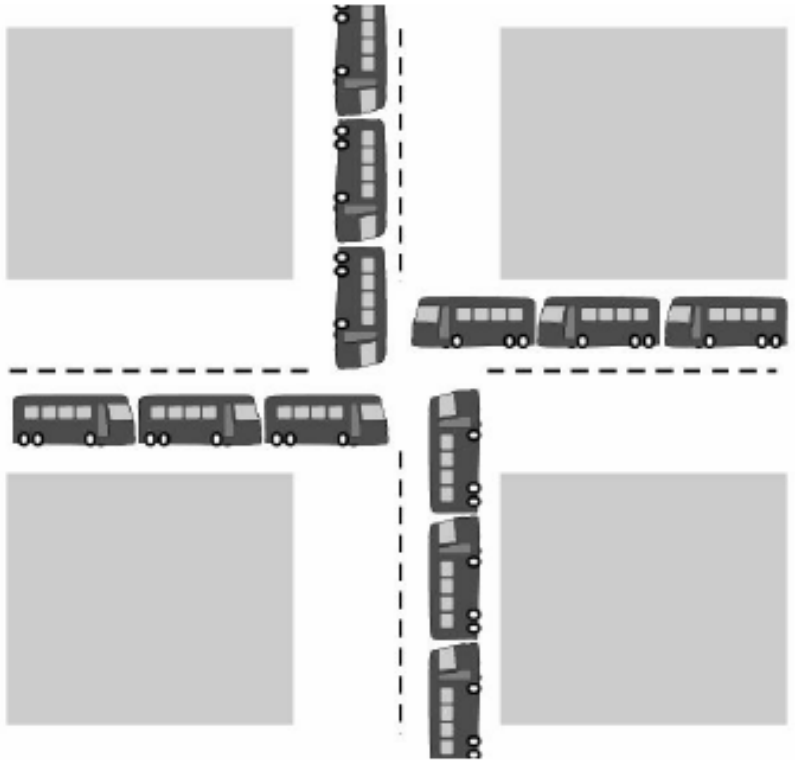
# Unit 6: Deadlocks

# Deadlock

A process in a multiprogramming system is said to be in ***deadlock*** if it is waiting for a particular event that will never occur.

# Deadlock Example

- All automobiles trying to cross.
- Traffic Completely stopped.
- Not possible without backing some.



A Traffic Deadlock

# Deadlock Scenarios

1. Let two processes each want to record a scanned document on a CD. Process A requests permission to use the scanner and is granted it. Process B is programmed differently and requests the CD recorder first and is also granted it. Now A asks for the CD recorder, but the request is denied until B releases it. Now, instead of releasing the CD recorder B asks for the scanner. At this point both processes are blocked and will remain so forever …**DEADLOCK**.
2. In a database system, a program may have to lock several records it is using, to avoid race conditions. If process A locks record R1 and process B locks R2, and then each process tries to lock the other one's record, we have a **DEADLOCK**.

# Resources

- Computer systems are full of resources that can only be used by one process at a time.
- Resource can be a hardware device (e.g. printer, tape drive, etc) or a piece of information (e.g. a locked record in a database).
- A process requests a resource before using it, and releases after using it.
  1. Request the resource.
  2. Use the resource.
  3. Release the resource.
- If the resource is not available when it is requested, the requesting process is forced to wait.

# Preemptable and Nonpreemptable Resources

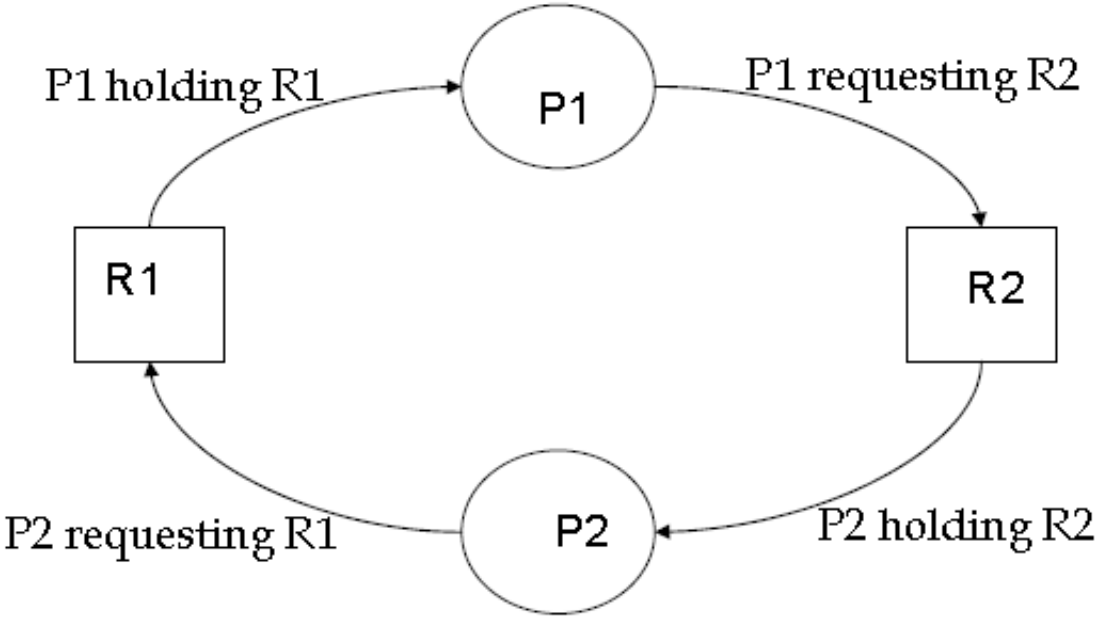
- A ***preemptable resource*** is one that can be taken away from the process owning it with no ill effects. For example, let us consider a system with 256 MB of user memory, one printer and two 256 MB processes that each want to print something. Process A requests and gets the printer, then starts to compute the values to print. Before it has finished with the computation, it exceeds its time quantum and is swapped out. Process B now runs and tries, unsuccessfully, to acquire the printer. This is a potential **deadlock** situation, because A has the printer and B has the memory, and neither one can proceed without the resource held by the other. However, it is possible to preempt (take away) the memory from B by swapping it out and swapping A in. Now, A can run, do its printing, and then release the printer. No deadlock occurs.
- A ***nonpreemptable resource*** is one that cannot be taken away from its current owner without causing the computation to fail. For example, if a process has begun to burn CD-ROM, suddenly taking the CD recorder away from it and giving it to another process will result in a garbled CD. So CD recorder is a nonpreemptable resource.

# Conditions for Resource Deadlocks

1. **Mutual Exclusion:** *Process claims exclusive control of resources they require.*
2. **Hold and Wait:** *Processes hold resources already allocated to them while waiting for additional resources.*
3. **No preemption:** *Resources previously granted can not be forcibly taken away from the process.*
4. **Circular wait:** *Each process holds one or more resources that are requested by next process in the chain.*

All four of these conditions must be present for a resource deadlock to occur. If one of them is absent, no resource deadlock is possible.

# Resource Deadlock




- Process P1 holds resource R1 and needs resource R2 to continue; Process P2 holds resource R2 and needs resource R1 to continue – deadlock.

*Key: Circular wait - deadlock*

# Deadlock Modeling

- The four deadlock conditions can be modeled using a directed graph.
- The graphs have two kinds of nodes: *processes*, denoted as *circles* and *resources*, denoted as *squares*.
- A directed arc from a resource node (square) to a process node (circle) means that the resource has previously been requested by, granted to, and is currently held by that process.
- A directed node from a process to a resource means that the process is currently blocked waiting for that resource.

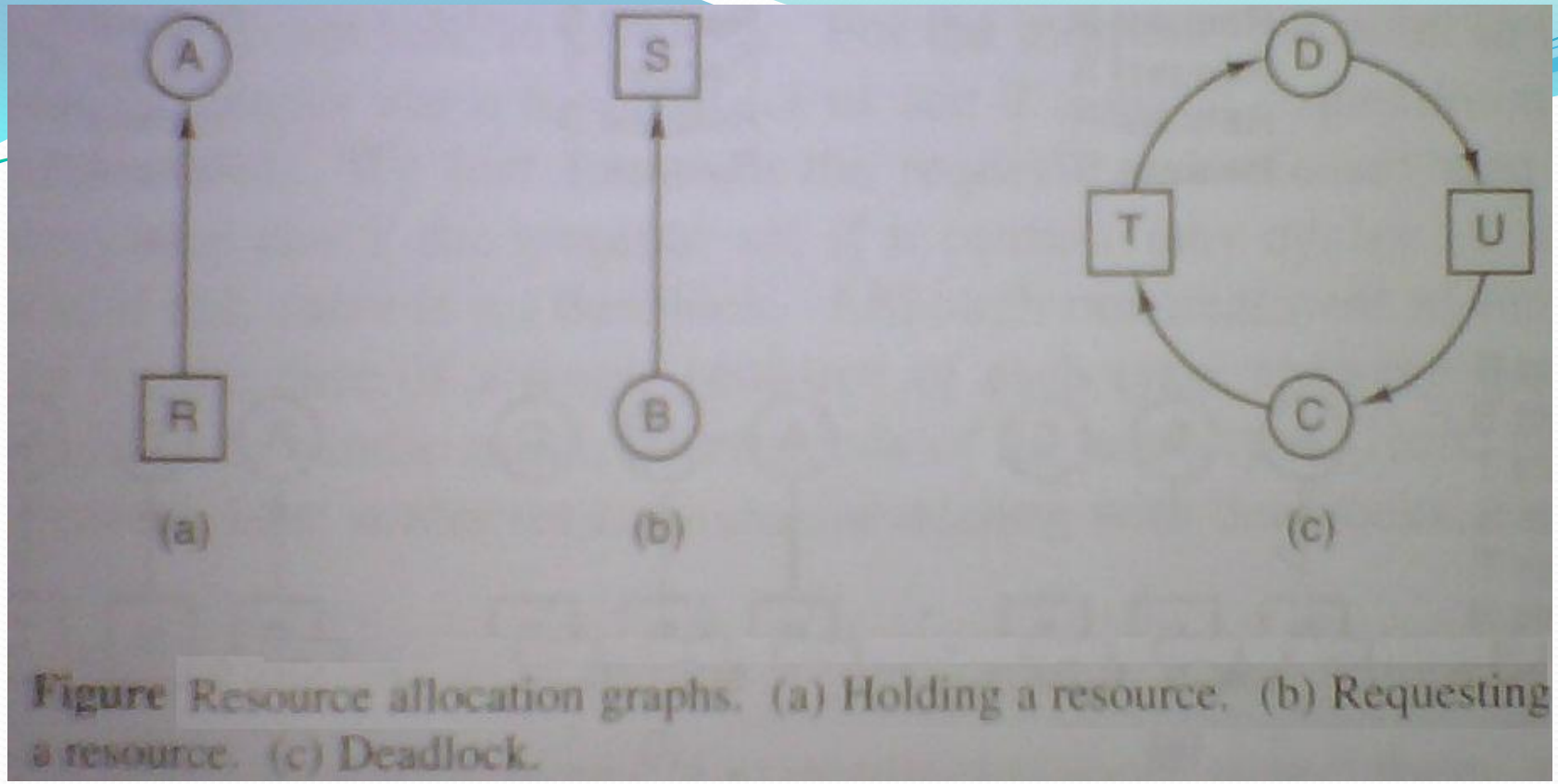



**Figure** Resource allocation graphs. (a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

- In fig (a), resource *R* is currently assigned to process A.
- In fig (b), process *B* is waiting for resource *S*.
- In fig (c), we see a deadlock: process *C* is waiting for resource *T*, which is currently held by process *D*. Process *D* is not about to release resource *T* because it is waiting for resource *U*, held by *C*. Both processes will wait forever. ***A cycle in the graph means that there is a deadlock involving the processes and resources in the cycle***. In this example, the cycle is *C*–*T*–*D*–*U*–*C*.

# Handling Deadlocks

Deadlock handling strategies:

1. ***Prevention:*** *We can use a protocol to prevent or avoid deadlocks, ensuring that the system never enters a deadlock state. This can be achieved by structurally negating one of the four required conditions.*
2. ***Dynamic Avoidance*** by careful resource allocation.
3. ***Detection and Recovery:*** *We can allow the system to enter a deadlock state, detect it, and recover.*
4. ***Just ignore the problem:*** *We can ignore the problem altogether, and pretend that deadlock will never occur in the system.*

# 1. Deadlock Prevention

***Key Idea:*** *If any one of the four necessary conditions for deadlock is denied, a deadlock can not occur.*

**Denying Mutual Exclusion:**

- If no resources were ever assigned exclusively to a single process , we would never have deadlocks.
- **Problem:** Some resources are strictly non-sharable (like printer), mutually exclusive control is required.
- *We can not prevent deadlock by denying the mutual exclusion.*

# Deadlock Prevention…

**Denying Hold and Wait:**

- ***Resources granted on all or none basis:*** All processes request all needed resources before starting execution. If all resources needed for processing are available, then it is granted and the process can run to completion. If complete set of resources is not available, the process must wait until available. While waiting, the process should not hold any resources.
- **Problem:** Low resource utilization and may lead to starvation. For example, let a process reads data from an input tape, analyzes it for an hour, and then writes an output tape as well as plotting the results. If all resources must be requested in advance, the process will tie up the output tape drive and the plotter for an hour.

# Deadlock Prevention…

**Denying No Preemption:**

- When a process holding resources is denied a request for additional resources, that process must release its held resources and if necessary, request them again together with additional resources.
- **Problem:** When a process releases resources, the process may lose all its work to that point. It may cause **indefinite postponement** or **starvation**.

# Deadlock Prevention…

**Denying Circular Wait:**

- All resources are uniquely numbered.
- **Rule:** Processes must request resources in linear ascending order. With this rule, the resource allocation graph can never have **cycles**.

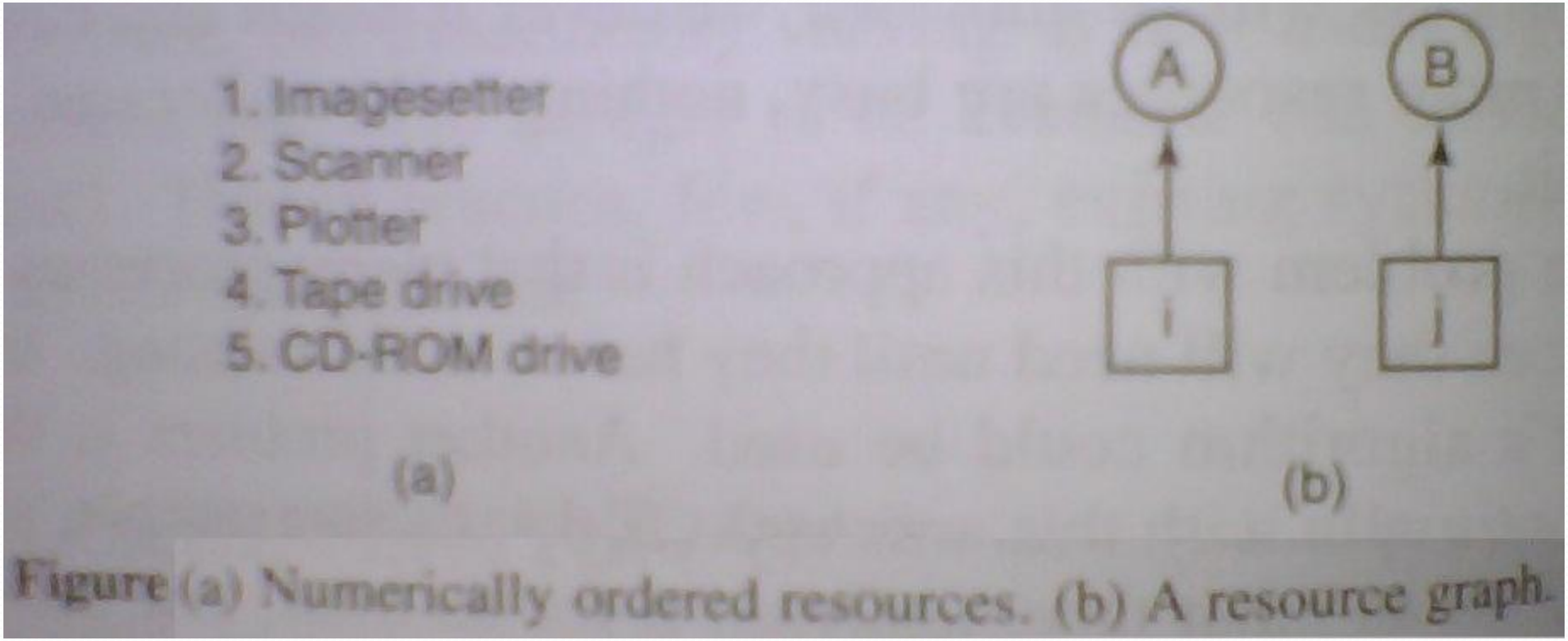



Figure (a) Numerically ordered resources. (b) A resource graph.

# Deadlock Prevention…

**Denying Circular Wait…**

Here, we can get a deadlock only if *A* requests resource *j* and *B* requests resource *i*. Since *i* and *j* are distinct resources, they will have different numbers. If $i > j$, then *A* is not allowed to request *j* because that is lower than what it already has. If $i < j$, then *B* is not allowed to request *i* because that is lower than what it already has. Either way, deadlock is impossible. The same logic holds with a number of processes.

- **Problem:** Difficult to maintain the resource order; dynamic update in addition of new resources. May also lead to **indefinite postponement** or **starvation**.

# 2. Deadlock Avoidance

- ***Key Idea:*** *Avoiding deadlock by careful resource allocation.*
- Decide whether granting a resource is **safe** or **unsafe**, and only make the allocation when it is **safe**.
- Needs extra information in advance: maximum number of resources of each type that a process may need.
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition.

# Deadlock Avoidance…

## Safe and Unsafe States

- A state is said to be ***safe*** if it is not deadlocked and there exists some scheduling order in which every process can run to completion even if all of the processes suddenly request their maximum number of resources immediately.
- **Example:** Suppose we have a state in which A has three instances of the resource but may need as many as nine eventually. B currently has two and may need four altogether, later. Similarly, C also has two but may need an additional five. A total of 10 instances of the resource exist, so with seven resources already allocated, there are three still free.

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 2 | 4 |
| C | 2 | 7 |

Free: 3

(a)

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 4 | 4 |
| C | 2 | 7 |

Free: 1

(b)

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 0 | – |
| C | 2 | 7 |

Free: 5

(c)

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 0 | – |
| C | 7 | 7 |

Free: 0

(d)

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 0 | – |
| C | 0 | – |

Free: 7

(e)

**Fig:** State in (a) is safe

| | Has | Max |
|---|---|---|
| A | 3 | 9 |
| B | 2 | 4 |
| C | 2 | 7 |

Free: 3

(a)

| | Has | Max |
|---|---|---|
| A | 4 | 9 |
| B | 2 | 4 |
| C | 2 | 7 |

Free: 2

(b)

| | Has | Max |
|---|---|---|
| A | 4 | 9 |
| B | 4 | 4 |
| C | 2 | 7 |

Free: 0

(c)

| | Has | Max |
|---|---|---|
| A | 4 | 9 |
| B | — | — |
| C | 2 | 7 |

Free: 4

(d)

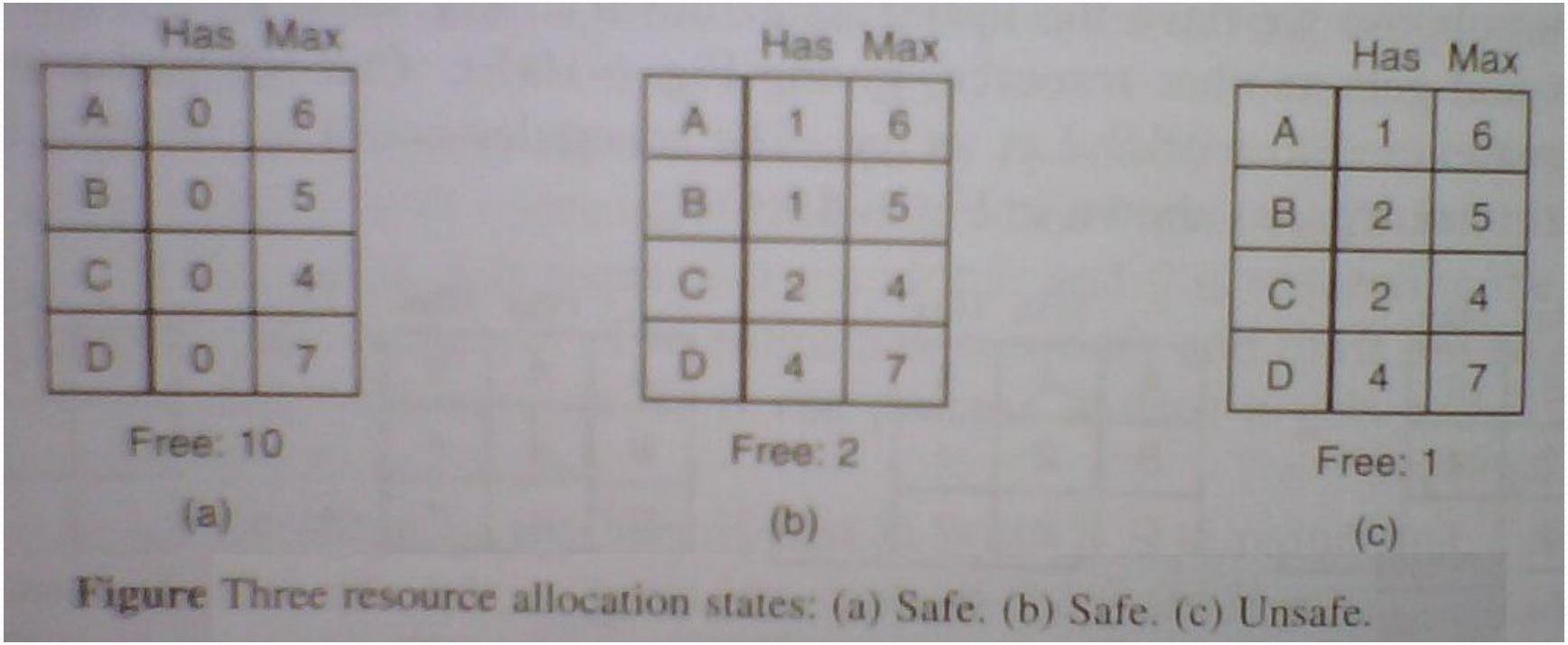
**Fig:** State in (b) is not safe.

In a safe state, system can guarantee that all processes will finish – no deadlock occur; from an unsafe state, no such guarantee can be given – deadlock may occur.

# Deadlock Avoidance…

## Banker's Algorithm

- Modeled on the way of banking system to ensure that the bank never allocates its available cash such that it can no longer satisfy the needs of all its customers.
- **Example:** Consider a small-town banker who deals with a group of customers to whom she has granted lines of credit. What the algorithm does is check to see if granting the request leads to an unsafe state. If it does, the request is denied and postponed until later. Let four customers A, B, C and D, each of whom have been granted a certain number of credit units (e.g. 1 unit = 1K dollars). The banker knows that not all customers will need their maximum credit immediately, so she has reserved only 10 units rather than 22 to service them. (**Here, customers = processes, units = resources like tape drives and banker = OS**). The customers make loan requests from time to time and at a certain instant, the situation is given in fig (b) below:



| | Has | Max |
|---|---|---|
| A | 0 | 6 |
| B | 0 | 5 |
| C | 0 | 4 |
| D | 0 | 7 |

Free: 10

(a)

| | Has | Max |
|---|---|---|
| A | 1 | 6 |
| B | 1 | 5 |
| C | 2 | 4 |
| D | 4 | 7 |

Free: 2

(b)

| | Has | Max |
|---|---|---|
| A | 1 | 6 |
| B | 2 | 5 |
| C | 2 | 4 |
| D | 4 | 7 |

Free: 1

(c)

**Figure** Three resource allocation states: (a) Safe. (b) Safe. (c) Unsafe.

The state in fig (b) is safe because with two units left, the banker can delay any request except C's, thus letting C finish and release all four of his resources. With four units in hand, the banker can let either D or B have the necessary units, and so on.

If a request from B for one more unit were granted ..... may lead to **DEADLOCK**.

# Banker's Algorithm…

- When a process requests the set of resources, the system determines whether the allocation of these resources will leave the system in safe state, if it will the resources are allocated, otherwise process must wait until some other process release enough resources.
- This algorithm utilizes the following data structures: **Available**, **Max**, **Allocation** and **Need**.
- Note: ***Need = Max – Allocation***

# Banker's Algorithm

**Safety Algorithm**

(1) $finish_i$ = FALSE for i = 0, 1, …,n

(2) Find an *i* such that

$need_i$ <= available && $finish_i$ = false

If no such i exists, goto step 4.

(3) available = available+ $allocation_i$;

$finish_i$= TRUE;

goto step 2;

(4) For each process

```
if (finish_i = True)
    printf("system is in safe state");
else
    printf("Unsafe state");
```

```
Resource_Request_Algorithm()
{
if(request_i <= need_i)
    if(request_i <= available)
            {
            available = available - request_i;
            allocation_i = allocation_i + request_i;
            need_i = need_i – request_i;
            }
    else
            wait;
else
    error;
}
```

# An Illustrative Example

- *Consider a system with five processes Po through $P_4$ and three resource types. Resource type A has 10 instances, resource type B has 5 instances, and resource type C has 7 instances. Suppose that, at some time, the following snapshot of the system has been taken.*

| | **Allocation** | **Max** | **Available** | **Need** |
|---|---|---|---|---|
| | A B C | A B C | A B C | A B C |
| Po | 0 1 0 | 7 5 3 | 3 3 2 | 7 4 3 |
| P1 | 2 0 0 | 3 2 2 | | 1 2 2 |
| P2 | 3 0 2 | 9 0 2 | | 6 0 0 |
| P3 | 2 1 1 | 2 2 2 | | 0 1 1 |
| P4 | 0 0 2 | 4 3 3 | | 4 3 1 |

Note: ***Need = Max - Allocation***

- The system is currently in safe state. The sequence <P1, P3, P4, P2, P0> satisfies the safety criteria.
- Now suppose that process P1 requests one instance of resource type A and two instances of resource type C, so $request_1$ = (1, 0, 2). Here, $\mathbf{request_1} <= \mathbf{need_1}$ (i.e. (1,0,2) <= (1,2,2)). Now to decide whether this request can be immediately granted, we check that $\mathbf{request_1} <= \mathbf{Available}$ (i.e. (1, 0, 2)<=(3, 3, 2)), which is **TRUE**.
- We then pretend that this request has been fulfilled, and arrive at the following new state:

| | **Allocation** | **Max** | **Available** | **Need** |
|---|---|---|---|---|
| | A B C | A B C | A B C | A B C |
| P0 | 0 1 0 | 7 5 3 | 2 3 0 | 7 4 3 |
| P1 | 3 0 2 | 3 2 2 | | 0 2 0 |
| P2 | 3 0 2 | 9 0 2 | | 6 0 0 |
| P3 | 2 1 1 | 2 2 2 | | 0 1 1 |
| P4 | 0 0 2 | 4 3 3 | | 4 3 1 |

- The next work is to determine whether this new system state is **safe**. To do so, we run our safety algorithm and find that the sequence <P1, P3, P4, P0, P2> satisfies our safety requirement. Hence, we can immediately grant the request of process P1.

## Classwork: Analyze it???

*(1) Can the request for (3, 3, 0) by $P_4$ be granted in this new state?*

*(2) Can a request for (0, 2, 0) by $P_0$ be granted in this new state?*

## Answers:

*(1) No, resources are not available.*

*(2) No, because even though resources are available, the resulting state will be unsafe.*

# Homework

A system has four processes and five allocatable resources. The current allocation and maximum needs are as follows:

| | Allocated | Maximum | Available |
|---|---|---|---|
| Process A | 1 0 2 1 1 | 1 1 2 1 3 | 0 0 **x** 1 1 |
| Process B | 2 0 1 1 0 | 2 2 2 1 0 | |
| Process C | 1 1 0 1 0 | 2 1 3 1 0 | |
| Process D | 1 1 1 1 0 | 1 1 2 2 1 | |

What is the smallest value of ***x*** for which this is a safe state?

***Answer*:** **[Hint: Use Need matrix]**

***x* = 2**

# Problem with Banker's Algorithm

- Algorithm requires fixed number of resources, however some processes dynamically change the number of resources required.
- Algorithm requires the number of resources in advance, however it is very difficult to predict the resources in advance.
- Algorithm supposes all processes return resources within finite time, but the system does not guarantee it.

# 3. Deadlock Detection and Recovery

- Instead of trying to prevent or avoid deadlock, system allows the deadlock to happen, and then recovers from deadlock when it does occur.
- *Hence, the mechanism for deadlock detection and recovery from deadlock is required.*

# Deadlock Detection with One Resource of Each Type

***Scenario:*** Consider a system with seven processes, *A* through *G*, and six resources, *R* through *W*. The state of which resources are currently owned and which ones are currently being requested is as follows:

- Process *A* holds *R* and wants *S*.
- Process *B* holds nothing but wants *T*.
- Process *C* holds nothing but wants *S*.
- Process *D* holds *U* and wants *S* and *T*.
- Process *E* holds *T* and wants *V*.
- Process *F* holds *W* and wants *S*.
- Process *G* holds *V* and wants *U*.

- Question: ***Is this system deadlocked, and if so, which processes are involved***???
- Solution: ***Construct resource graph and find out whether it contains a cycle.***

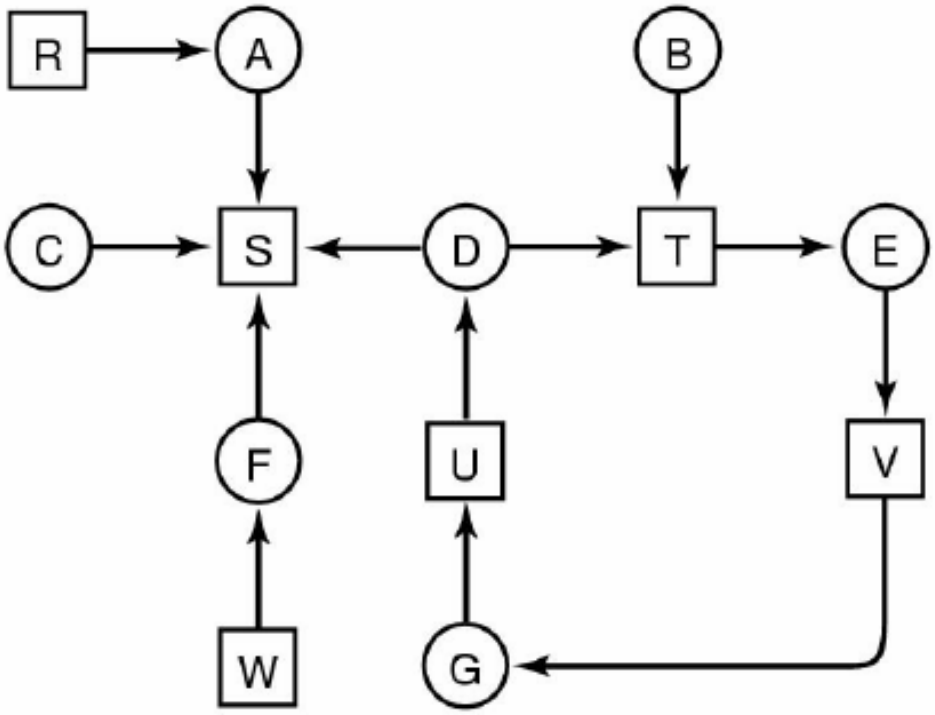



**Fig:** Resource graph

**Though it is possible to pick out the cycle by eye, how to detect cycle in directed graphs using an algorithm???**

# Algorithm

```
Deadlock_Detection_Algorithm(Resource_Graph_Input)
{
List L;
Boolean cycle = False;
    for each node
            L = Φ        /* initially empty list*/
            add node to L;
            for each node in L
                    for each arc
                            if (outgoing arc[i] == true)
                                    add corresponding node to L;
                                    if (it was already in list)
                                            cycle = true;
                                            print all nodes between these nodes;
                                            exit;
                            else                                    //if no such arc exists
                                    remove node from L;
                                    backtrack;
}
```

# Illustration

- Let us use the algorithm on the resource graph above. Let us start at R and then successively, *A*, *B*, *C*, *S*, *D*, *T*, *E*, *F*, and so forth. If we hit a cycle, the algorithm stops.
- We start at *R* and initialize *L* to the empty list. Then we add *R* to the list and move to the only possibility, *A*, and add it to *L*, giving *L* = [*R*, *A*]. From *A* we go to *S*, giving *L* = [*R*, *A*, *S*]. *S* has no outgoing arcs, so it is a dead end, forcing us to backtrack to *A*. Since *A* has no other outgoing arcs, we backtrack to *R*, completing our inspection of *R*.
- Now we restart the algorithm starting at *A*, resetting *L* to the empty list. This search, too, quickly stops, so we start again at *B*. From *B* we continue to follow outgoing arcs until we get to *D*, at which time *L* = [*B*, *T*, *E*, *V*, *G*, *U*, *D*]. Now if we pick *S*, we come to a dead end and backtrack to *D*. The second time we pick *T* and update *L* to be [*B*, *T*, *E*, *V*, *G*, *U*, *D*, *T*], at which point we discover the cycle and stop the algorithm.

## Deadlock Detection with Multiple Resources of Each Type

- When multiple copies of some of the resources exist, a different approach is needed to detect deadlocks.
- There exists a matrix-based algorithm for detecting deadlock among $n$ processes, $P_1$ through $P_n$.
- Let the number of resource classes be $m$, with $E_1$ resources of class 1, $E_2$ resources of class 2, and generally, $E_i$ resources of class $i$ ($1 <= i <= m$). $E$ is the **existing resource vector**. It gives the total number of instances of each resource in existence. For example, if resource class 1 is tape drives, then $E_1 = 2$ means the system has two tape drives.
- At any instant, some of the resources are assigned and are not available. Let $A$ be the **available resource vector**, with $A_i$ giving the number of instances of resource $i$ that are currently available (i.e., unassigned). If both of our two tape drives are assigned, $A_1$ will be 0.
- There are two arrays, $C$, the **current allocation matrix**, and $R$, the **request matrix**. The $i$-th row of $C$ tells how many instances of each resource class $P_i$ currently holds. Thus $C_{ij}$ is the number of instances of resource $j$ that are held by process $i$. Similarly, $R_{ij}$ is the number of instances of resource $j$ that $P_i$ wants. These four data structures are shown in below:

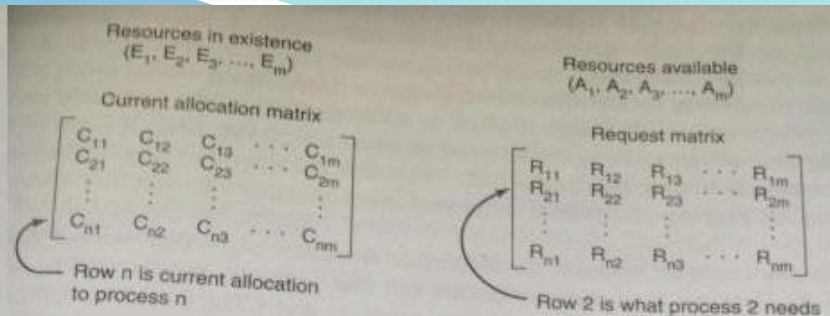



**Figure** The four data structures needed by the deadlock detection algorithm

An important invariant holds for these four data structures. In particular, every resource is either allocated or is available. This observation means that

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

In other words, if we add up all the instances of the resource $j$ that have been allocated and to this add all the instances that are available, the result is the number of instances of that resource class that exist.

The deadlock detection algorithm is based on comparing vectors. Let us define the relation $A \leq B$ on two vectors $A$ and $B$ to mean that each element of $A$ is less than or equal to the corresponding element of $B$. Mathematically, $A \leq B$ holds if and only if $A_i \leq B_i$ for $1 \leq i \leq m$.

Each process is initially said to be unmarked. As the algorithm progresses, processes will be marked, indicating that they are able to complete and are thus not deadlocked. When the algorithm terminates, any unmarked processes are known to be deadlocked. This algorithm assumes a worst-case scenario: all processes keep all acquired resources until they exit.

The deadlock detection algorithm can now be given as follows.

1. Look for an unmarked process, $P_i$, for which the $i$-th row of $R$ is less than or equal to $A$.
2. If such a process is found, add the $i$-th row of $C$ to $A$, mark the process, and go back to step 1.
3. If no such process exists, the algorithm terminates.

When the algorithm finishes, all the unmarked processes, if any, are deadlocked.

# Illustration

Let there be three processes and four resource classes, which we have labeled as tape drives, plotters, scanner, and CD-ROM drive. Process 1 has one scanner. Process 2 has two tape drives and a CD-ROM drive. Process 3 has a plotter and two scanners. Each process needs additional resources, as shown by the *R* matrix.

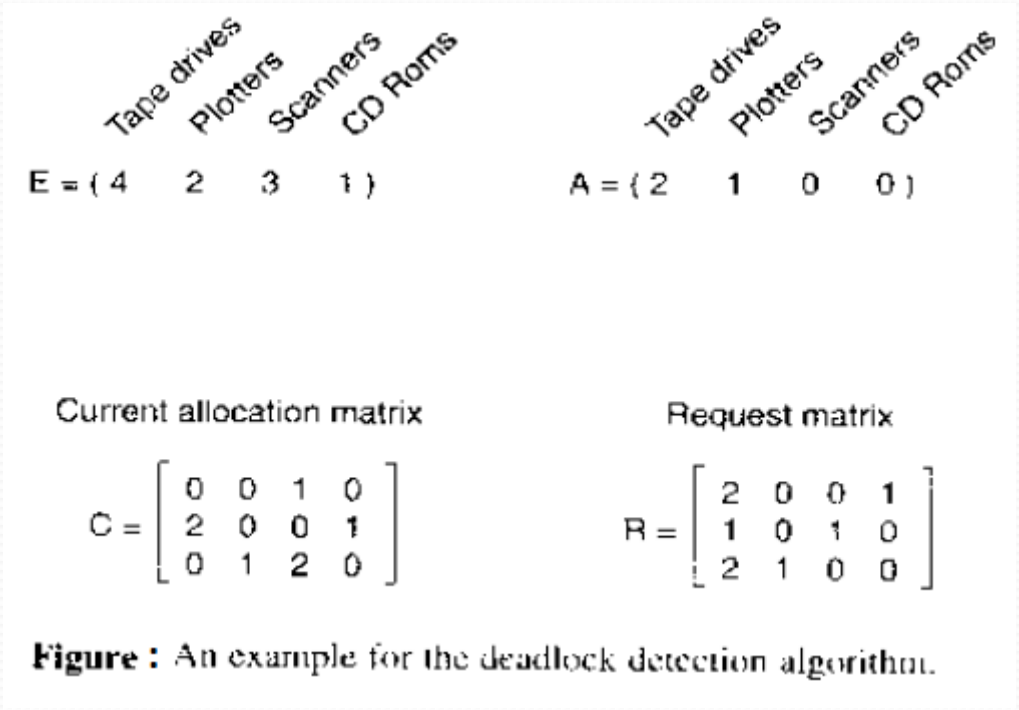



**Figure :** An example for the deadlock detection algorithm.

# Illustration…

To run the deadlock detection algorithm, we look for a process whose resource request can be satisfied. The first one cannot be satisfied because there is no CD-ROM drive available. The second cannot be satisfied either, because there is no scanner free. Fortunately, the third one can be satisfied, so process 3 runs and eventually returns all its resources, giving

$$A = (2\ 2\ 2\ 0)$$

At this point process 2 can run and return its resources, giving

$$A = (4\ 2\ 2\ 1)$$

Now the remaining process can run. There is no deadlock in the system.

Now consider a minor variation of the situation of above figure. Suppose that process 2 needs a CD-ROM drive as well as the two tape drives and the plotter. None of the requests can be satisfied, so the entire system is ***deadlocked***.

(Q) When to look for **deadlocks**?

(A1) Whenever a resource request is made … Very expensive in terms of CPU time.

(A2) Check every k minutes.

(A3) When the CPU utilization has dropped below some threshold. This is because if enough resources are deadlocked, there will be few runnable processes, and the CPU will often be idle.

# Recovery from Deadlock

- What to do next if the deadlock detection algorithm succeeds? –recover from deadlock.

# Recovery by Resource Preemption

- Preempt some resources temporarily from a process and give these resources to other processes until the deadlock cycle is broken.
- ***Problem:***
  - Depends on the resources.
  - Needs extra manipulation to suspend the process.
  - Very difficult.

# Recovery by Killing Processes

- Break deadlock by killing one or more processes in the cycle.
- A process not in the cycle can also be chosen as the victim in order to release its resources. For example, one process might hold a printer and want a plotter, with another process holding a plotter and wanting a printer. These two are **deadlocked**. A third process may hold another identical printer and another identical plotter and be happily running. Killing the third process will release these resources and break the deadlock involving the first two.

- **Problem:** If the process was in the midst of updating file, terminating it will leave file in incorrect state.
- ***Key idea:*** *We should terminate those processes, the termination of which incur the minimum cost.*

# 4. Ignore the Problem

- ***Fact:*** There is no good way of dealing with deadlock. So ignore the problem altogether.
- For most operating systems, deadlock is a rare occurrence. So the problem of deadlock is ignored just like an ostrich sticking its head in the sand and hoping the problem will go away.

- ***Note: Actually this bit of folklore is nonsense. Ostriches can run at 60 km/hour and their kick is powerful enough to kill any lion with visions of a big chicken dinner.***

# TU Exam Question 2068

What is deadlock? State the conditions necessary for deadlock to exist. Give reason, why all conditions are necessary.

# TU Exam Question 2067

List four necessary conditions for deadlock. Explain each of them briefly. What would be necessary (in the operating system) to prevent the deadlock.

# TU Exam Question 2066

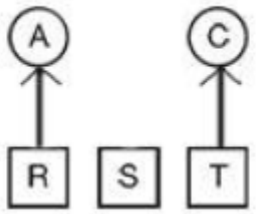
Define the term indefinite postponement. How does it differ from deadlock?

***Answer:*** ***Topic - Starvation on Page - 461 of textbook (Tanenbaum)***

# TU Exam Question 2066

A system has two processes and three identical resources. Each process needs a maximum of two resources. Is deadlock possible? Explain your answer.

***Answer:***




No, deadlock is not possible. If each process is allocated one resource, there is still third resource available, and whichever process takes it, that process will be able to run to completion.

# TU Model Question

A system has four processes P1 through P4 and two resource types R1 and R2. It has 2 units of R1 and 3 units of R2. Given that:

*P1 requests 2 units of R2 and 1 unit of R1*

*P2 holds 2 units of R1 and 1 unit of R2*

*P3 holds 1 unit of R1*

*P4 requests 1 unit of R1*

Show the resource graph for the state of the system. Is the system in deadlock, and if so, which processes are involved.